# MediCare Project

## 1. Project Planning & Management

## 1.1 Project Proposal

### 1.1.1 Project overview

MediCare is a web-based clinic management and appointment platform where patients book appointments with doctors, doctors manage their schedules and visits, and an admin monitors the whole system from a reporting dashboard.

**The platform has three types of users:**

- **Patient:** registers, searches for doctors by specialization, location and availability, books any doctor's open time slot, manages appointments, and views their medical records and prescriptions.
- **Doctor:** any doctor can create an account. A doctor sets working hours and vacation days, confirms or rejects appointment requests, records patient visits, and writes digital prescriptions.
- **Admin:** has a private page with daily reports and a dashboard, and manages doctors, patients, specializations and appointments.

A patient or doctor who registers for the first time must confirm their email address with a one-time password (OTP) sent to that address; the account becomes active only after the OTP is confirmed. The system sends emails (OTP and appointment notifications) through Gmail and does not use SMS.

MediCare is developed with ASP.NET Core MVC, Entity Framework Core and ASP.NET Identity, organised in an N-Tier architecture for separation of concerns, maintainability and scalability, and deployed on Microsoft Azure.

### 1.1.2 Problem statement

Traditional clinics often depend on manual appointment scheduling, phone calls and scattered patient records. This leads to double bookings, slow communication, difficulty finding a patient's medical history, and little visibility into how the clinic is performing.

MediCare addresses these problems with one central system: patients see which times are really available and book them directly, doctors control their own schedules and records, and the admin sees daily activity in one place.

### 1.1.3 Project objectives

The main objective is to build a clinic management and appointment system in which patients book appointments with doctors, doctors manage their schedules and patients, and an admin dashboard gives an overview of the whole system.

**The specific objectives are to:**

1. Let any doctor register an account, and let any patient book an appointment with any doctor, provided the chosen time slot is available.
2. Verify every new patient and doctor account with an email OTP before it is activated.
3. Let doctors manage working hours, vacation days and appointment requests.
4. Let doctors keep patient visit records and create digital prescriptions, and let patients view their history.
5. Give the admin a private page with daily reports, charts and management tools for doctors, patients, specializations and appointments.
6. Build the application with ASP.NET Core MVC and N-Tier architecture, using Entity Framework Core for data access.

### 1.1.4 Project scope

### In scope:

1. User management and email verification: three roles (Patient, Doctor, Admin), role-based registration and login, and email OTP confirmation at first registration for patients and doctors.
2. Doctor management and search: doctor profiles with specialization, consultation fee and profile image; patient search by specialization, location and availability.
3. Appointment management: patients book, cancel and reschedule only in time slots a doctor has made available; double bookings are prevented; doctors view their appointments, manage working hours and vacation days, and confirm or reject requests.
4. Patient records and prescriptions: doctors record visits, diagnoses, symptoms and treatment notes, and issue digital prescriptions with medication, dosage and frequency; patients view their medical and prescription history.
5. Email and in-app notifications: OTP, booking, confirmation, rejection and reminder emails sent through Gmail, plus real-time in-app alerts with SignalR.
6. Admin dashboard and daily reports: a private admin page with daily reports (new registrations, bookings, cancellations), charts, and management of doctors, patients, specializations and appointments, with export to PDF or Excel.
7. Application development: ASP.NET Core MVC, Entity Framework Core and ASP.NET Identity, organised with N-Tier architecture, Repository, Unit of Work and Factory patterns, and Dependency Injection.

## Out of scope:

- Integration with real hospital information systems or external electronic health record platforms.
- AI-based diagnosis, automated medical recommendations or clinical decision-making.
- Online payment processing, insurance claims and insurance-provider integration.
- SMS notifications; email through Gmail is the only external notification channel.
- Automatic verification of a doctor's medical licence with an official authority.
- Use of real patients' confidential medical data during development, testing or demonstration.
- Native Android or iOS apps; MediCare is a web application.
- Advanced financial accounting, inventory management and other full hospital-management features.

## 1.2 Project Plan

### 1.2.1 Overview:

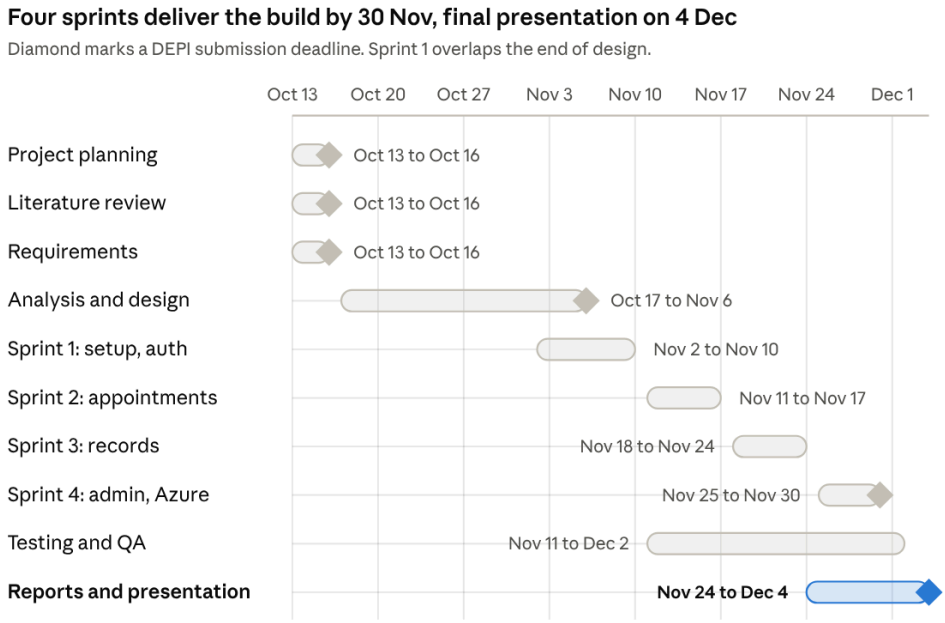
MediCare will be planned, built and handed in over 52 days, from 13 Oct to 4 Dec 2026, with the build split into four weekly sprints that follow the four-week scope in the project brief.

MediCare is an ASP.NET Core MVC clinic platform: patients search for doctors and book appointments, doctors manage schedules, medical records and prescriptions, and admins run the clinic from a dashboard with analytics. It uses N-Tier architecture (MediCare.Web, MediCare.Services, MediCare.Data) with the Repository, Unit of Work, Factory and Dependency Injection patterns, and is deployed on Microsoft Azure.

All dates follow the DEPI column of the submission schedule. All documents and code are uploaded to GitHub.

### 1.2.2 Timeline and Gantt chart:

The plan runs 13 Oct to 4 Dec: three planning phases in the first week, a three-week design phase, four build sprints and testing alongside them.




Gantt chart, 10 phases, 6 DEPI deadlines

Sprint 1 starts on 2 Nov, before design closes on 6 Nov, so setup overlaps with the last design week. Testing runs next to Sprints 2 to 4, and report writing starts in Sprint 3.

### 1.2.3 Milestones:

Eight milestones, four of them DEPI submission deadlines (16 Oct, 6 Nov, 30 Nov, 4 Dec); the other four are sprint and testing gates set by this plan.

| Milestones | Date | Milestone | Done when |
|---|---|---|---|
| M1 | Oct 16, 2026 | Planning, literature review and requirements submitted (DEPI deadline) | Proposal, plan, roles, risk plan and KPIs; lecturer feedback recorded; stakeholder analysis, user stories, functional and non-functional requirements on GitHub |
| M2 | Nov 6, 2026 | Analysis and design submitted (DEPI deadline) | Use case, ERD, schema, DFD, sequence, activity, state and class diagrams, architecture, wireframes, deployment and component diagrams |
| M3 | Nov 10, 2026 | Sprint 1 done: setup and user management | N-Tier solution, EF Core schema, repositories and Unit of Work, three-role authentication, doctor listing with search and filter |
| M4 | Nov 17, 2026 | Sprint 2 done: appointments and doctor dashboard | FullCalendar booking, cancel and reschedule, conflict detection, doctor dashboard, SignalR notifications |
| M5 | Nov 24, 2026 | Sprint 3 done: records, prescriptions, notifications | Medical records, printable prescriptions,  and SendGrid email, patient profile |
| M6 | Nov 30, 2026 | Implementation submitted (DEPI deadline) | Admin dashboard, Chart.js analytics with PDF/Excel export, responsive UI, live on Azure, README, CI/CD pipeline |
| M7 | Dec 2, 2026 | Testing complete | Test plan executed, bug report closed, no open critical bugs |
| M8 | Dec 4, 2026 | Final presentation and reports (DEPI deadline) | User manual, technical documentation, presentation, infographic, optional demo video |

### 1.2.4 Deliverables:

Every deliverable below is required by the DEPI documentation guidelines and goes to GitHub by its phase deadline.

| Phase | Deliverables | Due |
|---|---|---|
| Project planning and management | Project proposal, project plan (this document), task assignment and roles, risk assessment and mitigation plan, KPIs (for example response time, system uptime, user adoption rate) | 16 Oct |
| Literature review | Lecturer feedback and evaluation, suggested improvements, final grading criteria | 16 Oct |
| Requirements gathering | Stakeholder analysis, user stories and use cases, functional requirements, non-functional requirements | 16 Oct |
| System analysis and design | Problem statement and objectives, use case diagram and descriptions, software architecture (N-Tier), ER diagram, logical and physical schema, DFD, sequence, activity, state and class diagrams, wireframes and mockups, UI/UX guidelines, technology stack, deployment and component diagrams | 6 Nov |
| Implementation | Well-commented source code in a GitHub repository with branching strategy and commit history, CI/CD pipeline, README (installation, system requirements, configuration, execution guide), live deployment on Azure. Sprint-level features are in Milestones M3 to M6 | 30 Nov |
| Testing and QA | Test cases and test plan, automated tests (unit and integration), bug reports with resolutions, user acceptance testing plan | 4 Dec |
| Final presentation and reports | User manual, technical documentation (architecture, database schema, API documentation if applicable), presentation (PPT/PDF), project infographic, optional video demonstration | 4 Dec |

### 1.2.5 Resource allocation:

The work splits into five roles, one per layer of the N-Tier design plus management and quality.

| Role | Name | Responsibilities | Leads |
|---|---|---|---|
| Project manager | Dareen Ahmed Abdelfatah Yahia | Plan, sprint tracking, risk log, KPIs, GitHub organisation, final presentation | Planning, requirements, final reports |
| Backend developer | Philopatir Philip Raouf Farah<br>Youssef Ali Farghaly Mohamed | MediCare.Services business rules, Identity roles, appointment and conflict logic, prescriptions | Sprints 1 to 3 |
| Data and integrations developer | Ahmed Ali Mohamed | EF Core schema, repositories and Unit of Work, SignalR, SendGrid | Design (ERD), Sprints 1 and 3 |
| Frontend developer | Haneen Alaa Moustafa Mohamed<br>Dareen Ahmed Abdelfatah Yahia | MediCare.Web views, FullCalendar.js, Chart.js, Bootstrap 5, dark/light mode, accessibility | Design (wireframes), Sprints 2 and 4 |
| QA and DevOps engineer | All Team Members | Test plan and automated tests, bug reports, Azure App Service, Azure SQL, Azure SignalR Service, CI/CD | Testing, Sprint 4 deployment |

**Tools and services each phase depends on:**

| Resource | Used for | Needed by |
|---|---|---|
| GitHub | Source code, all documents, branching, pull requests, optional GitHub Actions pipeline | 13 Oct |
| ASP.NET Core MVC, Entity Framework Core, ASP.NET Identity, SQL database | Application, data access, authentication | 2 Nov |
| Microsoft Azure (App Service, Azure SQL, Azure SignalR Service) | Hosting and live deployment | 2 Nov (create resources), 30 Nov (live) |
| Azure DevOps or GitHub Actions | CI/CD pipeline | 11 Nov |
| SignalR, FullCalendar.js | Real-time notifications, booking calendar | 11 Nov |
| SendGrid | Email reminders | 18 Nov (accounts ready) |
| Chart.js, Bootstrap 5 | Analytics charts, responsive UI | 25 Nov |

## 1.3 Task Assignment & Roles

### 1.3.1 Team and roles:

Five team members each own one role, matched to the layers of the N-Tier design plus management and quality.

| Role | Name | Responsibilities | Leads |
|---|---|---|---|
| Project manager | Dareen Ahmed Abdelfatah Yahia | Plan, sprint tracking, risk log, KPIs, GitHub organisation, final presentation | Planning, requirements, final reports |
| Backend developer | Philopatir Philip Raouf Farah<br>Youssef Ali Farghaly Mohamed | MediCare.Services business rules, Identity roles, appointment and conflict logic, prescriptions | Sprints 1 to 3 |

| Role | Name | Responsibilities | Leads |
|---|---|---|---|
| Data and integrations developer | Ahmed Ali Mohamed | EF Core schema, repositories and Unit of Work, SignalR, , SendGrid | Design (ERD), Sprints 1 and 3 |
| Frontend developer | Haneen Alaa Moustafa Mohamed<br>Dareen Ahmed Abdelfatah Yahia | MediCare.Web views, FullCalendar.js, Chart.js, Bootstrap 5, dark/light mode, accessibility | Design (wireframes), Sprints 2 and 4 |
| QA and DevOps engineer | All Team Members | Test plan and automated tests, bug reports, Azure App Service, Azure SQL, Azure SignalR Service, CI/CD | Testing, Sprint 4 deployment |

### 1.3.2 Task assignment

Every task has one owner who is accountable for it and one or more supporters; dates match the project plan, and names are first names from the table above.

| Phase and dates | Task | Owner | Support |
|---|---|---|---|
| Planning, 13 to 16 Oct | Project proposal, project plan, KPIs, risk assessment, task assignment | Dareen | All |
| Planning, 13 to 16 Oct | Literature review: lecturer feedback, suggested improvements | Dareen | Philopatir |
| Requirements, 13 to 16 Oct | Stakeholder analysis, user stories and use cases | Dareen | Philopatir |
| Requirements, 13 to 16 Oct | Functional and non-functional requirements | Philopatir | Dareen, Youssef |
| Design, 17 Oct to 6 Nov | Problem statement, use case diagram, software architecture | Philopatir | Dareen |
| Design, 17 Oct to 6 Nov | ER diagram, logical and physical schema | Ahmed | Philopatir |
| Design, 17 Oct to 6 Nov | DFD, sequence, activity, state and class diagrams | Philopatir | Ahmed |
| Design, 17 Oct to 6 Nov | Wireframes, mockups, UI/UX guidelines | Haneen | Dareen |
| Design, 17 Oct to 6 Nov | Technology stack, deployment and component diagrams | Youssef | Ahmed |
| Sprint 1, 2 to 10 Nov | N-Tier solution, EF Core schema, repositories, Unit of Work | Ahmed | Philopatir |
| Sprint 1, 2 to 10 Nov | Multi-role authentication and registration (Patient, Doctor, Admin) | Philopatir | Haneen |
| Sprint 1, 2 to 10 Nov | Doctor listing with search and filter | Haneen | Philopatir |
| Sprint 1, 2 to 10 Nov | Azure resources and first CI pipeline | Youssef | Ahmed |
| Sprint 2, 11 to 17 Nov | Booking logic, cancel and reschedule, conflict detection | Philopatir | Youssef |
| Sprint 2, 11 to 17 Nov | FullCalendar booking screens and doctor dashboard | Haneen | Philopatir |
| Sprint 2, 11 to 17 Nov | SignalR real-time notifications | Ahmed | Haneen |
| Sprint 3, 18 to 24 Nov | Medical records and prescriptions logic | Philopatir | Ahmed |
| Sprint 3, 18 to 24 Nov | Records, prescription print view, patient profile screens | Haneen | Philopatir |
| Sprint 3, 18 to 24 Nov | SendGrid email, reminders, notification preferences | Ahmed | Philopatir |
| Sprint 4, 25 to 30 Nov | Admin dashboard logic | Philopatir | Haneen |
| Sprint 4, 25 to 30 Nov | Chart.js analytics, PDF and Excel export | Haneen | Ahmed |
| Sprint 4, 25 to 30 Nov | Responsive UI, dark/light mode, accessibility | Haneen | Dareen |
| Sprint 4, 25 to 30 Nov | Azure deployment, SignalR Service, CI/CD, README | Youssef | Ahmed |
| Testing, 11 Nov to 2 Dec | Test plan, automated tests, bug reports, user acceptance testing plan | Youssef | All fix their own bugs |
| Final reports, 24 Nov to 4 Dec | User manual | Dareen | Haneen |

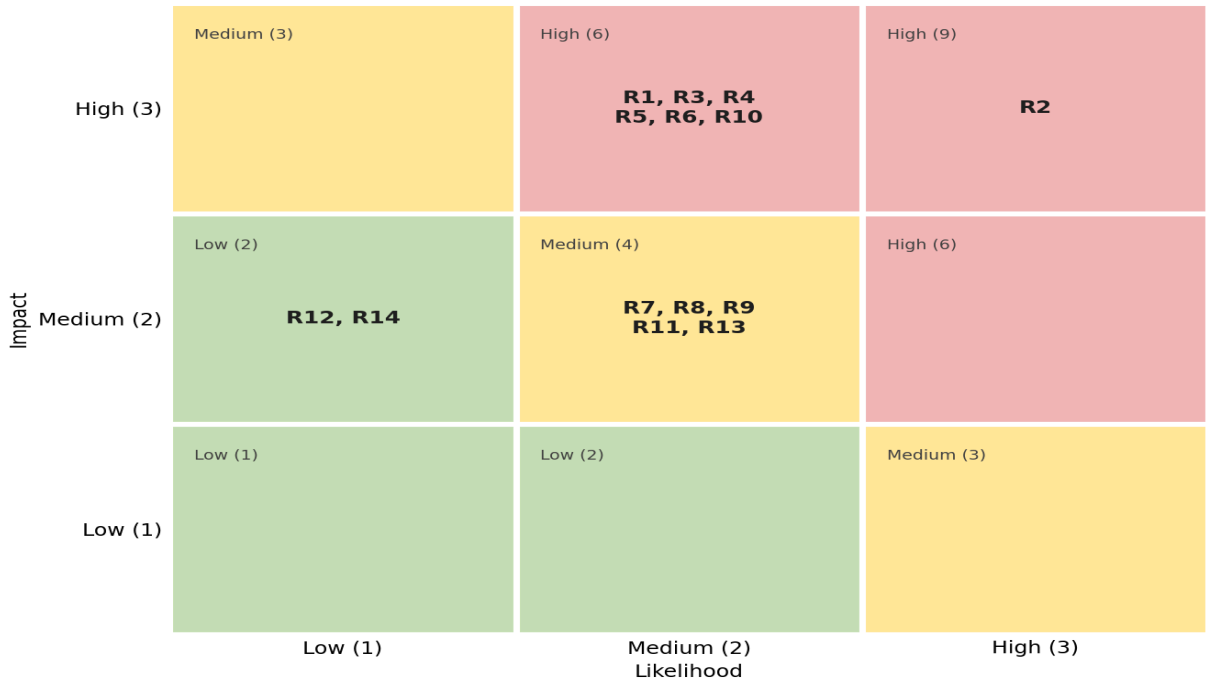
| Phase and dates | Task | Owner | Support |
|---|---|---|---|
| Final reports, 24 Nov to 4 Dec | Technical documentation (architecture, schema, API) | Philopatir | Ahmed |
| Final reports, 24 Nov to 4 Dec | Presentation, infographic, demo video | Dareen | All |

## 1.4 Risk Assessment & Mitigation Plan

We identified 14 risks: 7 rated High, 5 Medium and 2 Low. The High risks come from double bookings, email OTP delivery, protecting medical data, and the tight last sprint.

Level = likelihood x impact, with Low = 1, Medium = 2, High = 3. A score of 6 to 9 is High, 3 to 4 is Medium, 1 to 2 is Low. Owners are the team members from the Task Assignment document.

risk matrix · likelihood by impact, 14 risks




## Risk register

The register below gives the planned response and owner of each risk, from the highest level to the lowest.

| ID | Risk | Likeli-hood | Impact | Severity | Mitigation | Contingency | Owner |
|---|---|---|---|---|---|---|---|
| R1 | Design finishes late | Mediu m | High | High (6) | Freeze ERD and architecture by 30 Oct; start Sprint 1 setup on 2 Nov while wireframes finish. | Limit wireframes to Must-have screens; complete remaining mockups during Sprint 1. | Dareen |
| R2 | Sprint 4 overload | High | High | High (9) | Build daily report queries in Sprint 2; create Azure resources and first pipeline in Sprint 1; deploy at the end of every sprint. | Cut Should-have items first (export, dark/light mode). | Youssef |

| ID | Risk | Likeli-hood | Impact | Severity | Mitigation | Contingency | Owner |
|---|---|---|---|---|---|---|---|
| R3 | Double booking | Medium | High | High (6) | Unique doctor-and-slot constraint; check and save in one Unit of Work transaction; concurrency test cases. | Disable booking for the affected doctor; cancel the duplicate and notify the patient. | Philopatir |
| R4 | OTP or notification email not delivered | Medium | High | High (6) | SendGrid API key kept out of code; resend with cooldown; log send failures; test with a real inbox before each demo. | Use seeded demonstration accounts for demos; check the SendGrid account and sender settings. | Ahmed |
| R5 | OTP guessed or reused | Medium | High | High (6) | Six-digit code stored hashed, valid 10 minutes, one active code, locked after 5 wrong attempts. | Invalidate outstanding codes and require a new OTP; review logs. | Philopatir |
| R6 | Medical data exposed | Medium | High | High (6) | Role-based authorization on every page; patients see only their own records, doctors only their own patients; HTTPS; sample data only. | Disable the affected page; rotate secrets; review access logs. | Philopatir |
| R10 | Azure problems | Medium | High | High (6) | Confirm Azure access in Sprint 1; use the lowest suitable tier; README with a local-run guide as fallback. | Demonstrate locally following the README; record a demo video. | Youssef |
| R7 | Fake doctor accounts | Medium | Medium | Medium (4) | Listed only after email verification and a complete profile; admin can view, suspend or delete any doctor; licence check is future work. | Suspend the account and notify affected patients. | Philopatir |
| R8 | Team member unavailable | Medium | Medium | Medium (4) | Every task has an owner and a supporter; weekly sprint check; pull requests reviewed by a second member. | Reassign to the supporter; defer Should-have items. | Dareen |
| R9 | Merge conflicts | Medium | Medium | Medium (4) | Feature branches, protected main branch, pull requests, small daily commits. | Resolve in pair; short merge freeze. | Youssef |
| R11 | Wrong available slots | Medium | Medium | Medium (4) | Define slot rules during design; unit tests for availability, vacation days and rescheduling. | Doctor corrects schedule; hotfix release. | Philopatir |
| R13 | Scope creep | Medium | Medium | Medium (4) | The out-of-scope list is the limit; new ideas go to Future Enhancements. | Project manager decides; changes need team agreement. | Dareen |
| R12 | Slow dashboard | Low | Medium | Low (2) | Database indexes on doctor and date; paging on lists; daily totals computed in the database. | Restrict default date range. | Ahmed |
| R14 | Real-time alerts fail | Low | Medium | Low (2) | Use Azure SignalR Service; email stays the main channel so a missed alert is not a lost notification. | Alerts are also shown from stored notifications on next page load. | Ahmed |

## 1.5 KPIs (Key Performance Indicators)

MediCare is successful if it is fast, available, easy to adopt, free of double bookings and delivered on time; the 12 KPIs below turn each of those into a target that can be measured on the deployed system. The targets are the team's own proposals and can be adjusted after lecturer feedback.

| ID | Category | KPI | Proposed target | Measurement method | Result |
|---|---|---|---|---|---|
| KPI-01 | Performance | Page response time | 95% of page requests under 2 s | Azure metrics and browser developer tools | Not yet measured |
| KPI-02 | Performance | Booking speed | Under 2 min from search to booking | Timed task in user acceptance testing | Not yet measured |
| KPI-03 | Performance | OTP email delivery | 95% delivered within 1 min | Send log compared with test inbox | Not yet measured |
| KPI-04 | Performance | Admin dashboard load | Under 5 s with 10,000 sample appointments | Timed on the deployed system | Not yet measured |
| KPI-05 | Reliability | System uptime | 99% during the evaluation period ; | Azure availability monitor | Not yet measured |
| KPI-06 | Reliability | Double bookings | 0 duplicate active appointments per doctor slot | Concurrency tests | Not yet measured |
| KPI-07 | Security | Access control and secrets | 100% of role-access test cases pass; 0 secrets in the repository | Automated access tests; repository secret scan | Not yet measured |
| KPI-08 | Adoption | Sign-ups confirmed | 90% of started registrations confirmed with OTP | Registration vs. confirmation counts in test period | Not yet measured |
| KPI-09 | Adoption | Task completion | 80% of test users book or confirm without help | User acceptance sheet | Not yet measured |
| KPI-10 | Quality | Test cases passing | 90% passing; no critical bug open | Test plan and bug reports | Not yet measured |
| KPI-11 | Quality | Accessibility score | Lighthouse accessibility 90 or higher | Lighthouse at phone and desktop widths | Not yet measured |
| KPI-12 | Delivery | DEPI deadlines met | 4 of 4 deadlines; 90% of sprint tasks done | GitHub project board | Not yet measured |

# 2. Literature Review

This section records the lecturer's assessment, the team's suggested improvements, and how the project will be graded.

## 2.1 Feedback & evaluation:

The lecturer completes the feedback column after reviewing each submission, and the team writes its response next to it. The lecturer's assessment is not filled in by the team.

| Area reviewed | Lecturer's feedback | Team response |
|---|---|---|
| Project proposal and scope | Pending review | |
| Project plan and task assignment | Pending review | |
| Risk assessment and KPIs | Pending review | |
| Requirements gathering | Pending review | |
| System analysis and design | Pending review | |
| Implementation | Pending review | |
| Testing and quality assurance | Pending review | |
| Final presentation and reports | Pending review | |

## 2.2 Suggested improvements:

The team suggests eight enhancements that are outside the current scope but would make MediCare stronger; they are the first candidates if time allows or for a later version.

| **Improvement** | What it adds |
|---|---|
| **Doctor licence verification** | The admin checks a doctor's medical licence before the profile is listed, which closes the risk of fake doctor accounts (R7) |
| **OTP at every login for doctors and admin** | Stronger protection for accounts that can see or manage many patients' data |
| **Waiting list** | A patient joins a list for a fully booked doctor and gets an email when a slot opens |
| **Access log for medical records** | Records who opened which patient record and when, which supports privacy compliance |
| **Online payment of consultation fees** | Patients pay when booking; the fee is already stored on the doctor's profile |
| **Doctor ratings and reviews** | Helps patients choose between doctors of the same specialization |
| **Arabic and English interface** | Makes the platform usable for more patients in Egypt |
| **Video consultation** | Lets a doctor see patients remotely when a visit is not needed |

## 2.3 Final grading criteria:

The DEPI documentation guidelines name four graded areas but do not give the mark for each, so the marks column is left for the lecturer to confirm.

| Area | What is assessed | Deadline | Marks |
|---|---|---|---|
| Documentation | Project planning and management, literature review, requirements gathering, system analysis and design, all uploaded to GitHub | 16 Oct and 6 Nov | To be confirmed |
| Implementation | Structured and commented source code, coding standards, modular code, security and error handling, GitHub repository with branching and commit history, README, deployed application | 30 Nov | To be confirmed |
| Testing | Test cases and test plan, automated tests, bug reports and resolutions | 4 Dec | To be confirmed |
| Presentation and reports | User manual, technical documentation, project presentation, project infographic, optional video demonstration | 4 Dec | To be confirmed |

# 3. Requirements Gathering

MediCare has to satisfy three kinds of users and four supporting parties; their needs became the user stories, use cases, and functional and non-functional requirements below.

## 3.1 Stakeholder analysis:

| **Stakeholder** | **Type** | **Needs and expectations** | **Influence** | **Involvement** |
|---|---|---|---|---|
| Patients | Primary user | See only free slots; private records; email alerts. | High | Use the system daily; take part in user acceptance testing. |
| Doctors | Primary user | Control over hours and requests; quick patient history; prescriptions. | High | Use the system daily; define working-hours and prescription needs. |
| Admin | Primary user | Daily reports and dashboard; control of users and data. | High | Operates the private admin page. |
| Project team | Internal | Clear scope, a workable plan, accounts and tools. | High | Plan, build, test and document. |
| Lecturer and DEPI | Evaluator | Documents on GitHub, a working application, test evidence. | High | Reviews each submission and gives feedback and grades. |
| Email service (SendGrid) | External service | Valid account and sender settings; API key kept secret. | Medium | Delivers OTP and notification emails. |
| Hosting (Azure) | External service | Active account, correct setup, cost within budget. | Medium | Hosts the application, database and SignalR service. |

*Influence is the team's own rating. "Involvement" describes how each stakeholder is expected to take part in the project;*

## 3.2 User stories and use cases:

### 3.2.1 User stories:

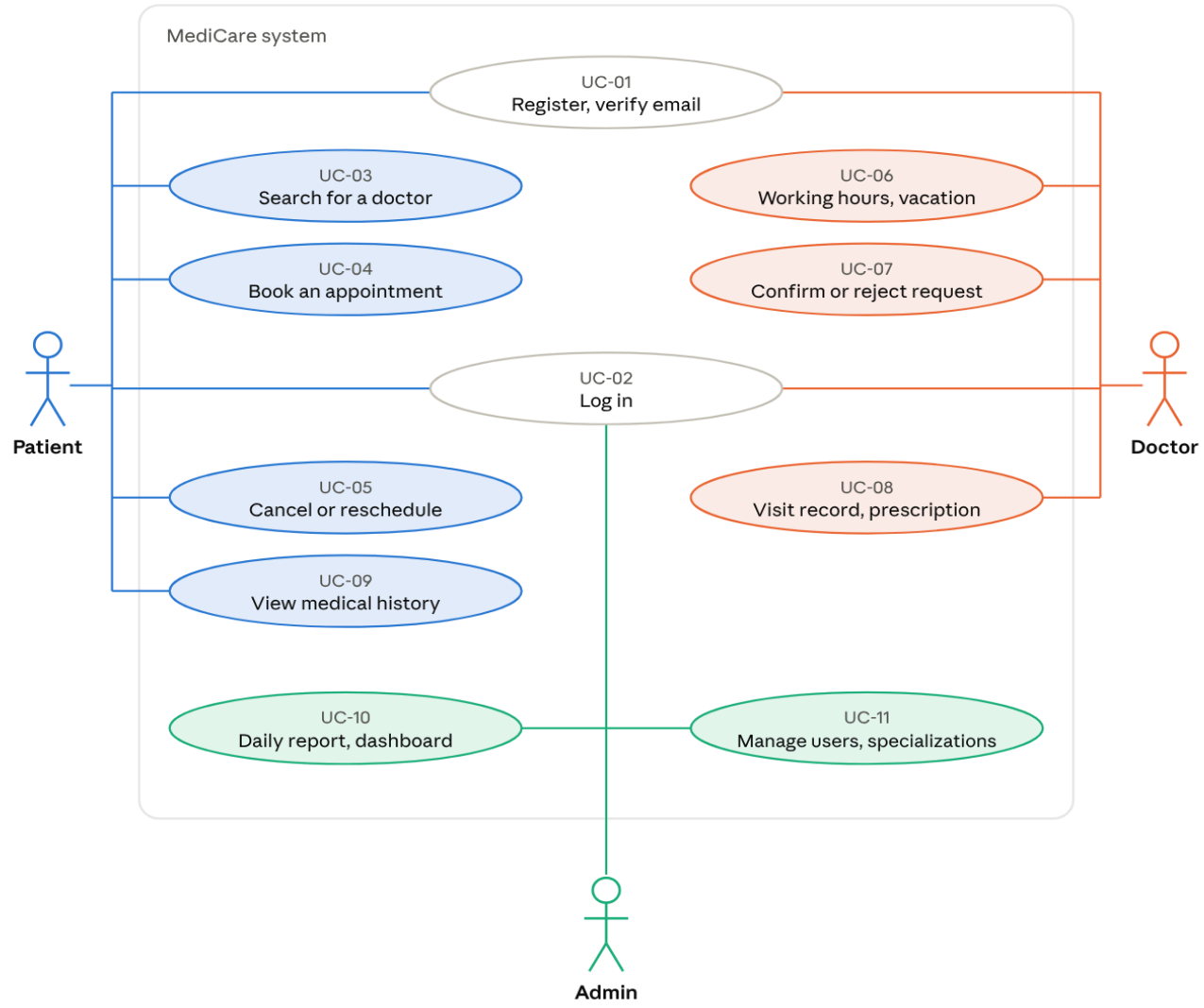
25 user stories cover the three roles: 10 for patients, 8 for doctors and 7 for the admin. 19 are Must-have and 6 are Should-have, which the team can cut first if the deadline is at risk.

| ID | As a | I want to | So that | Priority |
|---|---|---|---|---|
| US-01 | Patient | register with my email and confirm it with an OTP | my account is verified and secure | Must |
| US-02 | Patient | log in and log out securely | my data stays private | Must |
| US-03 | Patient | search for doctors by specialization, location and availability | I find the right doctor quickly | Must |
| US-04 | Patient | see a doctor's profile, fee and available time slots | I can choose a time that suits me | Must |
| US-05 | Patient | book an available time slot with any doctor | I get an appointment without phoning the clinic | Must |
| US-06 | Patient | cancel or reschedule my appointment | I can adapt when my plans change | Must |
| US-07 | Patient | see my upcoming and past appointments | I keep track of my visits | Must |
| US-08 | Patient | receive an email when my appointment is confirmed, rejected or close | I do not miss it | Must |
| US-09 | Patient | view my medical history and prescriptions | I can follow my treatment | Should |
| US-10 | Patient | update my personal details, allergies and medical history | my doctor sees correct information | Should |
| US-11 | Doctor | create an account and confirm my email with an OTP | I can start using the platform | Must |
| US-12 | Doctor | set my working hours and vacation days | patients can book only the times I can work | Must |
| US-13 | Doctor | see my daily and weekly appointments | I can plan my day | Must |
| US-14 | Doctor | confirm or reject appointment requests | I stay in control of my schedule | Must |

| ID | As a | I want to | So that | Priority |
|---|---|---|---|---|
| US-15 | Doctor | open a patient's information before the visit | I am prepared | Should |
| US-16 | Doctor | record the diagnosis, symptoms and treatment notes of a visit | the patient's history is complete | Must |
| US-17 | Doctor | write a digital prescription with medication, dosage and frequency | the patient gets clear instructions | Must |
| US-18 | Doctor | be alerted when a new appointment is booked | I can respond quickly | Should |
| US-19 | Admin | log in to a private admin page | only I can use admin tools | Must |
| US-20 | Admin | see a daily report of new registrations, bookings and cancellations | I follow what happens each day | Must |
| US-21 | Admin | see a dashboard with charts of appointments, popular doctors and specializations, and patient trends | I can make decisions from the data | Must |
| US-22 | Admin | view, suspend or delete doctor accounts | I can remove misuse | Must |
| US-23 | Admin | manage patients and specializations | the data stays clean and organised | Should |
| US-24 | Admin | view all appointments | I can oversee the whole system | Must |
| US-25 | Admin | export reports to PDF or Excel | I can share them | Should |

### 3.2.2 Use cases:

Three actors use eleven use cases: patients and doctors register with an email OTP, every user logs in, and each role then has its own use cases.




use case diagram · 3 actors, 11 use cases

## Use case descriptions

| | |
|---|---|
| **Use case** | UC-01 Register and verify email |
| **Actors** | Patient, Doctor |
| **Preconditions** | User is not registered and can access their email. |
| **Main flow** | 1. User chooses a role and opens the registration form.<br>2. User enters role details (doctor: specialization, location, fee, image; patient: personal details, allergies, medical history) and accepts the privacy notice.<br>3. System validates the data and creates an inactive account.<br>4. System emails a 6-digit OTP.<br>5. User enters the OTP.<br>6. System verifies the code and activates the account. |

| | |
|---|---|
| **Alternative flows** | A1 Email already used or data invalid: system shows an error. A2 Wrong OTP: attempt is counted; after 5 wrong attempts the code is locked. A3 OTP expired (10 minutes): user requests a new OTP after the cooldown. A4 Email not received: user requests a resend. |
| **Postconditions** | Account is active; doctors are listed in the directory once the profile is complete; inactive accounts cannot log in. |

| | |
|---|---|
| **Use case** | UC-02 Log in |
| **Actors** | Patient, Doctor, Admin |
| **Preconditions** | Account is active. |
| **Main flow** | 1. User enters email and password.<br>2. System authenticates the user.<br>3. System opens the home page of the user's role and applies role restrictions. |
| **Alternative flows** | A1 Invalid credentials: generic error. A2 Account unverified or suspended: login refused with a message. A3 User logs out: session ends. |
| **Postconditions** | An authenticated session exists for the role. |

| | |
|---|---|
| **Use case** | UC-03 Search for a doctor |
| **Actors** | Patient |
| **Preconditions** | Patient is logged in. |
| **Main flow** | 1. Patient enters search criteria (specialization, location, availability).<br>2. System lists matching doctors with fee and location.<br>3. Patient opens a doctor profile and views free slots. |
| **Alternative flows** | A1 No match: system shows an empty-result message. A2 Patient clears filters and searches again. |
| **Postconditions** | No data is changed. |

| | |
|---|---|
| **Use case** | UC-04 Book an appointment |
| **Actors** | Patient (Doctor notified) |
| **Preconditions** | Patient is logged in; the doctor has at least one free slot. |
| **Main flow** | 1. Patient selects a free slot in the calendar and confirms.<br>2. System re-checks availability and creates the appointment as Pending in one transaction.<br>3. System notifies the doctor in-app and sends confirmation emails as configured.<br>4. System shows the booking result. |
| **Alternative flows** | A1 Slot taken in the meantime: system refuses and refreshes the free slots. A2 Doctor on vacation or outside working hours: slot is not offered. |
| **Postconditions** | Appointment is Pending; the slot is held; the doctor is notified. |

| | |
|---|---|
| **Use case** | UC-05 Cancel or reschedule an appointment |

| | |
|---|---|
| **Actors** | Patient (Doctor notified) |
| **Preconditions** | Patient has a Pending or Confirmed future appointment. |
| **Main flow** | 1. Patient opens My appointments (upcoming and past).<br>2. Patient selects an appointment and chooses Cancel or Reschedule.<br>3. Cancel: system sets Cancelled, releases the slot and emails the patient and doctor.<br>4. Reschedule: patient selects a new free slot; system checks it, moves the appointment and releases the old slot. |
| **Alternative flows** | A1 Appointment is past, Rejected, Cancelled or Completed: action not allowed. A2 New slot taken: system asks for another. A3 Cut-off time and status after rescheduling. |
| **Postconditions** | Appointment is Cancelled or moved; old slot is free; notifications are sent. |

| | |
|---|---|
| **Use case** | UC-06 Manage working hours and vacation days |
| **Actors** | Doctor |
| **Preconditions** | Doctor is logged in. |
| **Main flow** | 1. Doctor opens the schedule page.<br>2. Doctor defines weekly working hours and vacation days.<br>3. System validates and saves, and recalculates free slots. |
| **Alternative flows** | A1 Invalid or overlapping times: error shown. A2 Vacation day overlaps existing appointments: handling . A3 Doctor edits own profile (specialization, fee, photo). |
| **Postconditions** | Free slots reflect the new schedule; profile changes are visible in the directory. |

| | |
|---|---|
| **Use case** | UC-07 Confirm or reject a request |
| **Actors** | Doctor (Patient notified) |
| **Preconditions** | Doctor is logged in; at least one Pending request exists. |
| **Main flow** | 1. Doctor views daily or weekly appointments in calendar or list view.<br>2. Doctor opens a Pending request.<br>3. Doctor confirms or rejects.<br>4. System updates the status and notifies the patient by email and in-app alert. |
| **Alternative flows** | A1 Patient cancelled meanwhile: system shows the request as Cancelled. A2 Rejected: slot is released. |
| **Postconditions** | Status is Confirmed or Rejected; patient is notified. |

| | |
|---|---|
| **Use case** | UC-08 Record visit and issue prescription |

| | |
|---|---|
| **Actors** | Doctor |
| **Preconditions** | Doctor is logged in; the appointment is Confirmed and belongs to the doctor. |
| **Main flow** | 1. Doctor opens the patient's information.<br>2. Doctor records diagnosis, symptoms and treatment notes.<br>3. Doctor optionally adds prescription lines (medication, dosage, frequency).<br>4. System saves the visit and prescription in one transaction and marks the appointment Completed.<br>5. Doctor may open the print view. |
| **Alternative flows** | A1 Validation error: system highlights the fields. A2 No prescription needed: step 3 is skipped. A3 Editing a saved record . |
| **Postconditions** | Visit record and prescription are saved and visible to that patient only (and the treating doctor). |

| | |
|---|---|
| **Use case** | UC-09 View medical history |
| **Actors** | Patient |
| **Preconditions** | Patient is logged in. |
| **Main flow** | 1. Patient opens Medical history.<br>2. System lists the patient's own visits.<br>3. Patient opens a visit and views notes and prescriptions. |
| **Alternative flows** | A1 No records: system shows an empty-state message. A2 Patient updates personal details, allergies and medical history. A3 Request for another user's record: refused. |
| **Postconditions** | No clinical data is changed by the patient. |

| | |
|---|---|
| **Use case** | UC-10 View daily report and dashboard |
| **Actors** | Admin |
| **Preconditions** | Admin is logged in. |
| **Main flow** | 1. Admin opens the private admin page.<br>2. Admin selects a date; system shows the daily report.<br>3. Admin views dashboard charts.<br>4. Admin optionally exports the report to PDF or Excel. |
| **Alternative flows** | A1 No data for the date: report shows zeros. A2 Export fails: error message; admin retries. |
| **Postconditions** | No data is changed; an exported file may be produced. |

| | |
|---|---|
| **Use case** | UC-11 Manage users and specializations |
| **Actors** | Admin |
| **Preconditions** | Admin is logged in. |

| | |
|---|---|
| **Main flow** | 1. Admin opens the management lists (doctors, patients, specializations, appointments).<br>2. Admin views, edits, suspends or deletes doctors and patients, and adds, edits or deletes specializations.<br>3. System saves the change. |
| **Alternative flows** | A1 Deletion of an account or specialization that is in use: handling. |
| **Postconditions** | Changes are saved; suspended accounts cannot log in. |

## 3.3 Functional requirements:

The system has 35 functional requirements in six areas: 30 are Must-have and 5 are Should-have.

| ID | Area | The system shall | Priority |
|---|---|---|---|
| FR-01 | Accounts | support three roles: Patient, Doctor and Admin | Must |
| FR-02 | Accounts | let a patient or a doctor register with name, email, password and role details (doctor: specialization, location, consultation fee, profile image; patient: personal information, allergies, medical history) | Must |
| FR-03 | Accounts | send a 6-digit OTP to the email address at first registration and activate the account only after the correct OTP is entered | Must |
| FR-04 | Accounts | keep each OTP valid for 10 minutes, store it hashed, lock it after 5 wrong attempts, and allow a new OTP to be requested after a short cool down | Must |
| FR-05 | Accounts | let users log in and out with email and password, and restrict every page to the roles allowed to use it | Must |
| FR-06 | Accounts | create the Admin account during system setup, not through public registration | Must |
| FR-07 | Doctors | list a doctor in the doctor directory once registration and email verification are complete | Must |
| FR-08 | Doctors | show on each doctor profile the specialization, consultation fee, location, photo and available slots | Must |
| FR-09 | Doctors | let patients search and filter doctors by specialization, location and availability | Must |
| FR-10 | Doctors | let a doctor edit their own profile | Must |
| FR-11 | Appointments | let a doctor set weekly working hours and vacation days | Must |
| FR-12 | Appointments | show a patient only free slots, meaning inside working hours, outside vacation days and not already booked, in an interactive calendar | Must |
| FR-13 | Appointments | let a patient book a free slot with any doctor | Must |
| FR-14 | Appointments | prevent double booking: one slot holds one active appointment, even when two patients book at the same moment | Must |
| FR-15 | Appointments | track each appointment status: Pending, Confirmed, Rejected, Cancelled or Completed | Must |
| FR-16 | Appointments | let a patient cancel or reschedule their own appointment and release the old slot | Must |
| FR-17 | Appointments | let a doctor confirm or reject pending requests | Must |
| FR-18 | Appointments | let a doctor see daily and weekly appointments in calendar and list views | Must |
| FR-19 | Appointments | let a patient see upcoming and past appointments | Must |

| ID | Area | The system shall | Priority |
|---|---|---|---|
| FR-20 | Records | let a doctor record the diagnosis, symptoms and treatment notes of a visit | Must |
| FR-21 | Records | let a doctor create a digital prescription with medication, dosage and frequency | Must |
| FR-22 | Records | provide a print-friendly prescription view | Must |
| FR-23 | Records | let a patient view their own visit history and prescriptions, and no one else's | Must |
| FR-24 | Records | let a patient update their personal details, allergies and medical history | Should |
| FR-25 | Records | let a doctor open the information of patients who have an appointment with them | Should |
| FR-26 | Notifications | send emails through Gmail for OTP, booking received, confirmed, rejected, cancelled and rescheduled | Must |
| FR-27 | Notifications | send a reminder email 24 hours before a confirmed appointment | Should |
| FR-28 | Notifications | show real-time in-app alerts to the doctor for a new booking and to the patient for a confirmation or rejection | Should |
| FR-29 | Admin | provide a private admin page available only to the Admin role | Must |
| FR-30 | Admin | show a daily report for a chosen date: new patient and doctor registrations, appointments booked, confirmed and cancelled | Must |
| FR-31 | Admin | show a dashboard with charts of monthly appointments, most popular doctors and specializations, and patient trends | Must |
| FR-32 | Admin | let the admin view, edit, suspend or delete doctors and patients | Must |
| FR-33 | Admin | let the admin add, edit and delete specializations | Must |
| FR-34 | Admin | let the admin view all appointments | Must |
| FR-35 | Admin | export reports to PDF or Excel | Should |

## 3.4 Non-functional requirements:

The system has 19 non-functional requirements in seven categories; the performance, uptime and usability figures match the KPIs in section 3.

| ID | Category | Requirement | Measure |
|---|---|---|---|
| NFR-01 | Performance | Pages respond quickly under normal use | 95% of page requests in under 2 seconds |
| NFR-02 | Performance | The admin dashboard stays fast as data grows | Loads in under 5 seconds with 10,000 sample appointment records |
| NFR-03 | Performance | OTP emails reach the user promptly | 95% delivered within 1 minute |
| NFR-04 | Security | Passwords are never stored in plain text | Stored as hashes by ASP.NET Identity |
| NFR-05 | Security | All traffic is encrypted | The deployed site is reachable only over HTTPS |

| ID | Category | Requirement | Measure |
|---|---|---|---|
| NFR-06 | Security | Access rules are enforced on the server, not only by hiding buttons | A patient or doctor who opens another user's record by URL is refused |
| NFR-07 | Security | The application resists common web attacks | Input validation, anti-forgery tokens on forms, output encoding, parameterised queries through Entity Framework Core |
| NFR-08 | Security | Secrets are kept out of the source code | Gmail app password and database connection string live in configuration or Azure settings, never in GitHub |
| NFR-09 | Privacy | Users agree to how their data is used | Registration requires accepting a privacy notice; only data the features need is collected |
| NFR-10 | Usability | The interface works on phones and desktops | Responsive layout built with Bootstrap 5, checked at phone and desktop widths |
| NFR-11 | Usability | A new patient can book without help | Booking completed in under 2 minutes in user acceptance testing |
| NFR-12 | Usability | The interface is accessible | Lighthouse accessibility score of at least 90; form labels, keyboard navigation and readable contrast; dark and light mode |
| NFR-13 | Usability | Errors are clear | Every form shows a plain validation message next to the field that failed |
| NFR-14 | Reliability | The system is available when needed | At least 99% uptime during the evaluation period |
| NFR-15 | Reliability | Related changes are saved together or not at all | Booking, cancelling and prescribing use one Unit of Work transaction |
| NFR-16 | Reliability | Failures do not expose technical details | A friendly error page is shown and the error is logged |

| ID | Category | Requirement | Measure |
|---|---|---|---|
| NFR-17 | Maintainability | Layers are separated | Web, Services and Data projects; Repository and Unit of Work patterns; Dependency Injection; commented code that follows naming standards |
| NFR-18 | Maintainability | Work is traceable | GitHub repository with feature branches, pull requests and meaningful commit messages; README with setup and run steps |
| NFR-19 | Compatibility | The site works in common browsers | Latest versions of Chrome, Edge and Firefox |